Trabajo Práctico N°1

IA 5.2 Computer Vision

1° Cuatrimestre 2026

Sistema de Reconocimiento Facial

1. Introducción

El objetivo de este trabajo práctico es consolidar y aplicar los conocimientos adquiridos en las primeras unidades de la materia de *Computer Vision*, integrando conceptos teóricos con la implementación de un sistema real de reconocimiento facial.

Se espera que el estudiante no solo comprenda **cómo construir un sistema**, sino también **por qué se toman determinadas decisiones de diseño**, analizando los trade-offs entre precisión, costo computacional y complejidad del sistema.

El pipeline implementado deberá seguir el enfoque estándar de sistemas modernos:

Detección → Alineación → Embeddings → Comparación

El siguiente enlace provisto corresponde a una función del backend que forma parte del pipeline (servicio de procesamiento facial) y resulta relevante para comprender la integración del sistema:

Deberá realizarse un fork del repositorio provisto por la cátedra como template del trabajo practico.

---

2. Objetivo

Desarrollar un sistema completo de **reconocimiento e identificación facial**, capaz de:

- Detectar rostros en imágenes o video
- Extraer representaciones (embeddings)
- Identificar o verificar personas conocidas
- Manejar casos de desconocidos

Alcances

Repositorio base

El estudiante deberá realizar un **fork del** repositorio **provisto por la cátedra**.

3. Alcance del sistema

El trabajo deberá cubrir el pipeline completo de un sistema moderno de reconocimiento facial:

1. **Detección de rostros**
2. **Alineación facial** (normalización geométrica)
3. **Extracción de embeddings**
4. **Identificación / verificación**

Se recomienda desacoplar cada etapa para facilitar testing, debugging y extensibilidad.

Restricciones de implementación

La cátedra proveerá la arquitectura completa (frontend + backend + base de datos).

El estudiante deberá implementar o reemplazar las siguientes funciones:

- align_face
- detect_faces
- extract_embedding_from_face

Cualquier modificación adicional en el backend deberá estar **debidamente justificada**.

4. Alcance del trabajo práctico

El trabajo deberá incluir en la notebook de python train.ipynb la documentacion necesaria de los siguientes elementos :

Dataset

Se deberá:

- Construir un dataset propio (familiares, amigos, etc.)
- Complementar con datasets públicos o utilizados por la cátedra

Recomendaciones:

- Variar condiciones (iluminación, pose, expresión)
- Mantener balance de clases
- Documentar el dataset (cantidad de imágenes por persona, fuente, etc.)

Preprocesamiento

Definir y justificar técnicas como:

- Normalización (resize, mean/std)
- Data augmentation (flip, blur, ruido, etc.)
- Filtrado de imágenes de baja calidad

Modelo (Backbone)

Elegir y justificar una arquitectura:

- CNNs (ResNet, EfficientNet, etc.)
- Transformers (ViT, DeiT)
- Híbridos

Se evaluará:

- Justificación técnica y clara de la elección del modelo utilizado para desplegar la aplicación :
    - modelo pre-entrenado + fine tuning
    - modelos públicos disponibles pre-entrenados
- Trade-offs (performance vs costo computacional)

Embeddings y base vectorial

Construir una base de embeddings con estructura similar a:

```
id_imagen: str

embedding: list[float]

path: str

etiqueta: str

metadata: dict[str, Any] = Field(default_factory=dict)
```

La base se utilizará para almacenar embeddings generados por los modelos y respectiva metadata generada al registrar personas en el sistema.

Opciones:

- JSON (simple)
- O utilizar Base de datos vectoriales:
    - PostgreSQL + pgvector **(provista por la catedra)**
    - MongoDB
    - ChromaDB

Se debe implementar búsqueda por similitud (cosine similarity, L2, etc.).

Backend (API)

Se deberá implementar las funciones mencionadas anteriormente para dejar un backend funcional que:

- Exponga endpoints:

Endpoint	Descripción
`/insert`	Registrar nueva identidad
`/predict`	Inferencia sobre imagen
`/status`	Consultar estado asincrónico

Requisitos:

- Procesamiento asincrónico
- Respuesta con HTTP 202 + UUID

Formato de `/status`:

```
{
  "status": "done | inProgress | failed",
  "link": "url | none"
}
```

El `link` deberá permitir descargar la imagen/video una vez procesado con:

- Bounding boxes
- Keypoints
- Etiquetas

Interfaz de usuario

Se utilizará el frontend provisto (Gradio).

Input:

- Imagen

Output:

- Bounding boxes
- Keypoints faciales
- Nombre + score de similitud

Configuración

No se permite hardcodear configuraciones.

Debe utilizarse:

- Variables de entorno
- Archivos `.env`

Ejemplos:

- Threshold de similitud
- Modelo utilizado
- Paths

5. Entregables

Los alumnos deberán entregar un repositorio propio utilizando como base el template provisto por la cátedra, con todo lo mencionado en las secciones previas.

Para que el trabajo práctico se considere aprobado, el sistema debe andar sin errores corriendo los siguientes comandos :

docker compose build

docker compose up

Se debe entregar un enlace con el pull request que será evaluado por la cátedra sin mergear a main o el branch principal.

---

6. Buenas prácticas esperadas

Se valorará especialmente:

- Código modular (separación por capas)
- Manejo de errores
- Logging
- Tests básicos (opcional pero recomendado)
- Uso de tipado (type hints)

Documentación

Debe incluirse en la notebook de train :

- Explicación del pipeline
- Justificación del modelo elegido
- Proceso de fine-tuning
- Elección de hiper parámetros
- Problemas encontrados y soluciones

---

8. Evaluación

Se evaluará:

- **Funcionalidad** (demo obligatoria)
- **Calidad del código**
- **Arquitectura del sistema**
- **Manejo de errores**
- **Documentación**
- **Justificación técnica**
- **Participación (GitHub commits + PRs)**
- **Defensa oral**

---

9. Integrantes

Máximo 2 personas.

---

10. Fecha de entrega

Viernes 08/05 hasta las 23:59 (vía campus).

- Solo se considerarán commits hasta esa hora
- Se recomienda trabajar de forma incremental (no un único commit final)

---

11. Recomendaciones finales (clave para aprender bien)

- No copiar implementaciones sin entenderlas
- Visualizar embeddings (PCA / t-SNE) para validar separabilidad
- Testear con datos fuera del dataset
- Analizar falsos positivos / falsos negativos
- Medir métricas básicas (accuracy, precision, recall)